

HGDL: Methodology and Implementation

1 October 2026 — describes `hgd1` 2.4 plus the changes listed as *Unreleased* in `CHANGELOG.md`

Abstract

HGDL (Hybrid Global Deflated Local) searches a differentiable function for many optima at once and returns a growing, sorted list of *unique* optima rather than a single answer. It combines distributed local optimizations (“walkers”), deflation of every optimum already found so that it is not found again, and a global (genetic) step that places new walkers. This document describes the mathematics of each component, how several deflations are combined, how constraints are treated, and how the method is implemented and executed asynchronously on a disk cluster. Pseudocode follows the code closely; Section 10 maps every concept to the function that implements it.

Contents

1	Problem and notation	2
2	Overview	3
3	Local search: the walker	3
4	Deflation	4
4.1	One bump	4
4.2	Several deflations: the product operator	4
4.3	Behaviour near a deflated point	4
4.4	How deflation acts on the different local optimizers	5
4.5	The deflation radius	6
5	Accepting and classifying results	6
5.1	Unconstrained problems	6
5.2	Classification	6
5.3	The arrival test	7
6	Constraints	7
6.1	Constraint representation	7
6.2	First-order conditions: multipliers by bounded least squares	7
6.3	Second-order conditions and the radius	8
6.4	Newton polish on the KKT system	8
6.5	Constraints and deflation	8
7	Global step	9

8 Asynchronous execution	9
8.1 Coordinator and walkers	9
8.2 Consistency between walkers and the changing deflation set	9
8.3 The first results	10
8.4 Stopping and time limits	10
8.5 Reproducibility	10
9 Pseudocode	11
10 Implementation map	14
11 Constants	14
12 Differences from HGDN and open issues	14

1 Problem and notation

HGDL considers

$$\min_{x \in \Omega} f(x) \quad \text{subject to} \quad c^L \leq c(x) \leq c^U, \quad \Omega = \{x \in \mathbb{R}^D : l \leq x \leq u\}, \quad (1)$$

where $f : \mathbb{R}^D \rightarrow \mathbb{R}$ is twice differentiable with gradient $g = \nabla f$ and Hessian $H = \nabla^2 f$, and the (optional) constraints $c : \mathbb{R}^D \rightarrow \mathbb{R}^m$ have Jacobian J ; an equality constraint has $c_i^L = c_i^U$, a one-sided constraint an infinite bound. The user supplies f and g ; H is optional and otherwise replaced by forward differences of g (step 10^{-6} , upper triangle mirrored).

Instead of a single minimizer, HGDL returns a list

$$\mathcal{L} = \{(x_k, f(x_k), \text{class}_k, \lambda_k, g_k, r_k)\}_k,$$

sorted by $f(x_k)$, where λ_k are curvature eigenvalues, g_k the gradient (of the Lagrangian under constraints) and r_k the deflation radius. Two modes decide what is sought:

minimization (default) strict local minima, with any local optimizer;

stationary points all non-degenerate stationary points (minima, maxima, saddle points), with the built-in Newton method and without constraints.

The list is never truncated, because it doubles as the set of deflation points: the subset

$$\mathcal{D} = \{(x_k, r_k) \in \mathcal{L} : \text{class}_k \in \{\text{minimum, maximum, saddle point}\}\}$$

is deflated (Section 4). The user-facing limit `number_of_optima` only limits how many entries `get_latest()/get_final()` return.

The box Ω is the domain from which starting points are drawn. It is *not* treated as a constraint in the sense of (1): bound-respecting scipy optimizers keep their iterates in Ω , the built-in Newton method only projects onto it, and users are advised to choose wide bounds.

2 Overview

A *walker* is one local optimization started from a point x^0 . HGDL keeps one walker running on every worker thread of a dask cluster. Whenever a walker finishes:

1. its result is judged: is it an acceptable optimum (Section 5), and is it new, i.e. not inside the deflation radius of a known optimum?
2. if so, it is classified, inserted into $\mathcal{L}$ and, from then on, deflated;
3. a new walker is started, from a point proposed by the global step (Section 7), and it sees the deflation set as it is at that moment.

The run ends after $N_{\text{epochs}} \times N_w$ walkers have finished, where N_w is the number of walkers at the start (the number of worker threads). There is no synchronization between walkers (Section 8); the name `num_epochs` is kept from the earlier, epoch-synchronized design and now only sets the length of the run.

Every walker optimizes a *deflated* problem: the gradient and Hessian it sees are modified so that they blow up near every known optimum (Section 4). Which results are accepted, however, is always decided on the *true* derivatives.

3 Local search: the walker

A walker receives a start x^0 , the problem, and a snapshot of the deflation set $\{(\hat{x}_k, r_k)\}_{k=1}^K$. It builds the deflated gradient G and deflated Jacobian J_G (Section 4) and calls one of three local optimizers:

dNewton HGDL’s Newton iteration on $G(x) = 0$ (Algorithm 3). In stationary-point mode it takes the plain Newton step $s = -J_G^{-1}G$ (least squares if J_G is singular) and converges to any nearby stationary point. In minimization mode it takes the *saddle-free* step [2]

$$s = -V \text{diag}(1/\max(|\theta_i|, \varphi))V^T G, \quad \text{sym}(J_G) = \frac{1}{2}(J_G + J_G^T) = V \text{diag}(\theta)V^T, \quad (2)$$

with the floor $\varphi = 10^{-8} \max(1, \max_i |\theta_i|)$. Because $|\text{sym}(J_G)|$ is positive definite, s is a descent direction and the iteration is repelled by maxima and saddle points. Iterates are projected onto Ω ; the iteration stops when both the step and G are below the tolerance, after `local_max_iter` (default 1000) iterations, or on a non-finite step.

scipy any method of `scipy.optimize.minimize`, called with the *true* objective f , the deflated gradient G as `jac` and J_G as `hess`, the bounds and constraints. With constraints, HGDL always uses SLSQP.

callable a user function `f(func, grad, hess, bounds, x0, *args)` returning a scipy-like result; it receives the deflated derivatives as well.

The walker then judges its own result (Section 5) and returns $(x, f(x), g, \lambda, r, \text{success})$.

4 Deflation

4.1 One bump

Deflation follows the HGDN method of Noack and Funke [1]. Around a known optimum $\hat{x}$ with radius r , the bump function (Eq. (7) of [1] with radial support, $\alpha = r^2$) is

$$\beta(x; \hat{x}, r) = \begin{cases} \exp(1 - 1/a(x)), & a(x) = 1 - \|x - \hat{x}\|^2 / r^2 > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

It is C^∞ , equal to 1 at $\hat{x}$, strictly between 0 and 1 inside the ball $\|x - \hat{x}\| < r$, and 0 outside. Its gradient is

$$\nabla \beta(x) = -\frac{2\beta(x)}{a(x)^2 r^2} (x - \hat{x}). \quad (4)$$

4.2 Several deflations: the product operator

With K deflation points the *deflation operator* is the product

$$\mathcal{D}(x) = \prod_{k=1}^K \frac{1}{1 - \bar{\beta}_k(x)}, \quad \bar{\beta}_k = \min(\beta(x; \hat{x}_k, r_k), \beta_{\max}), \quad \beta_{\max} = 1 - 10^{-5}, \quad (5)$$

and the deflated gradient and its Jacobian are

$$G(x) = \mathcal{D}(x) g(x), \quad (6)$$

$$J_G(x) = \mathcal{D}(x) H(x) + g(x) \nabla \mathcal{D}(x)^T, \quad \nabla \mathcal{D} = \mathcal{D} \sum_{k=1}^K \frac{\nabla \beta_k}{1 - \bar{\beta}_k}. \quad (7)$$

(In Eq. (7) the code uses $\nabla \beta_k$ of the uncapped bump; the cap only keeps the denominator away from zero.) J_G is the exact Jacobian of G and is not symmetric.

Each factor of (5) is ≥ 1 , so $\mathcal{D} \geq 1$ everywhere and $\mathcal{D} = 1$ outside all balls: deflation only rescales the gradient, it never changes its direction. Overlapping balls simply multiply. An additive operator $1/(1 - \sum_k \beta_k)$ would not be safe: where two bumps overlap, $\sum_k \beta_k$ can exceed 1, the operator changes sign, and a deflated region turns into an attracting one. Figure 1(a) shows both. Because all K bumps are evaluated together (vectorized over the deflation points), one evaluation of $\mathcal{D}$ costs $O(KD)$.

4.3 Behaviour near a deflated point

Let $e = x - \hat{x}$, $d = \|e\| \ll r$, and assume one bump is active and $g(\hat{x}) = 0$. From $1/a = 1 + d^2/r^2 + O(d^4/r^4)$ we get $\beta = \exp(-d^2/r^2 + O(d^4/r^4))$ and therefore

$$1 - \beta \approx \frac{d^2}{r^2}, \quad \mathcal{D} \approx \frac{r^2}{d^2}, \quad \nabla \ln \mathcal{D} \approx -\frac{2e}{d^2}, \quad g \approx He. \quad (8)$$

Consequently $\|G\| \approx (r^2/d^2) \|He\| \geq r^2 |\lambda|_{\min}/d \rightarrow \infty$: the root of g at $\hat{x}$ is removed (Figure 1(b)).

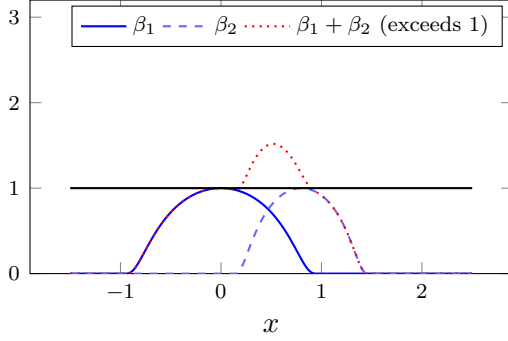
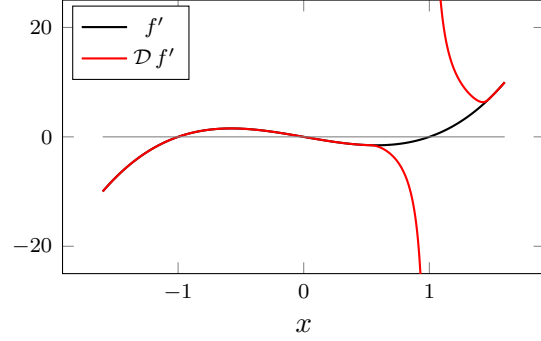
(a) two bumps, $\hat{x} = 0$, $r = 1$ and $\hat{x} = 0.8$, $r = 0.7$ (b) $f'(x)$ and $\mathcal{D}f'$ for $f = (x^2 - 1)^2$, $\hat{x} = 1$, $r = 0.5$ 

Figure 1: (a) Bumps (3) of two overlapping deflation balls. Their sum exceeds 1 where they overlap, which would flip the sign of an additive operator; the product operator (5) stays ≥ 1 . (b) The derivative of the double well and its deflated version with the root $\hat{x} = 1$ deflated: away from $\hat{x}$ the two coincide (the roots -1 and 0 are untouched), near $\hat{x}$ the deflated derivative grows without bound instead of crossing zero. The radius 0.5 is enlarged for visibility (HGDL would use $1/f''(1) = 1/8$).

The Newton step moves away from $\hat{x}$. The Newton equation $J_G s = -G$ is, after dividing by $\mathcal{D}$, $(H + g \nabla \ln \mathcal{D}^T) s = -g$. Inserting (8),

$$H \left(I - \frac{2ee^T}{d^2} \right) s = -He.$$

The matrix $R = I - 2ee^T/d^2$ is a Householder reflection ($R^2 = I$, $Re = -e$), so $s = -Re = e$: the Newton step goes from $x = \hat{x} + e$ to $\hat{x} + 2e$, straight away from the deflated point, doubling its distance. This is the mechanism by which deflated Newton walkers leave the neighbourhood of known solutions; we verified it numerically with HGDL's implementation of (6)–(7) ($s/e = 0.9999$ at $d = 10^{-2}r$).

The cap. With $\beta_{\max} = 1 - 10^{-5}$, $\mathcal{D} \leq 10^5$ per bump; by (8) the cap is reached for $d \lesssim r\sqrt{10^{-5}} \approx 3 \cdot 10^{-3}r$. The cap keeps $\mathcal{D}$ finite, but it also means that $G(\hat{x}) = \mathcal{D}(\hat{x})g(\hat{x}) = 0$: $\hat{x}$ remains a root of G inside a tiny ball. A walker that lands there is rejected by the arrival test of Section 5.3.

4.4 How deflation acts on the different local optimizers

Deflation changes only the derivatives, never f , and G is a positive multiple of g . Whether a walker started in the basin of a deflated minimum leaves that basin therefore depends on the local optimizer:

- **Plain Newton** (stationary-point mode) steps away from $\hat{x}$, as shown above.
- **Saddle-free Newton** (minimization mode): $|\text{sym}(J_G)|$ is positive definite, so the step (2) satisfies $g^T s < 0$ and is a descent direction *for f itself*. Near $\hat{x}$, $J_G \approx \mathcal{D}HR$ with the reflection R above, so $e^T \text{sym}(J_G)e \approx -\mathcal{D}e^T H e < 0$: deflation creates negative curvature along e . Taking absolute values of the eigenvalues turns the step from $+e$ into approximately $-e$ (numerically $s^T e / \|e\|^2 = -0.99$): the walker is pulled back to $\hat{x}$.

Table 1: Where 100 walkers started uniformly in $(0.05, 1.95)^2$, the basin of the deflated minimum $(1, 1)$ of the four-well function, end up (radius $r = 1/8$).

mode	optimizer	back at $(1, 1)$	elsewhere stationary	not converged
stationary points	plain Newton	1	99	0
minimization	L-BFGS-B	0	34	66
minimization	saddle-free Newton	100	0	0

- **Line-search quasi-Newton methods** (L-BFGS-B, SLSQP, ...) evaluate the true f but the rescaled gradient G . The inconsistent pair distorts their step lengths and curvature updates; walkers leave the deflated ball but often stall without converging.

Table 1 quantifies this on the four-well function $f = (x_1^2 - 1)^2 + (x_2^2 - 1)^2$ with only the minimum $(1, 1)$ deflated and 100 walkers started in its basin. In minimization mode, uniqueness of the reported minima therefore rests on the arrival test, not on deflation; Section 12 discusses a remedy for the saddle-free step.

4.5 The deflation radius

An accepted point x with curvature eigenvalues λ (Section 5) gets

$$r = \min\left(\frac{1}{\lambda_*}, \rho \|u - l\|\right), \quad \lambda_* = \begin{cases} \min_i \lambda_i & \text{minimization,} \\ \min_i |\lambda_i| & \text{stationary points,} \end{cases} \quad \rho = 0.1. \quad (9)$$

At a stationary point the principal curvatures of the graph of f are exactly the Hessian eigenvalues, so $1/\lambda_*$ is the largest principal radius of curvature (the osculating circle in the flattest direction) — an estimate of the extent of the optimum. Like every curvature of the graph it depends on the scale of f ; this is deliberate. The cap keeps a single poorly conditioned optimum from deflating the whole domain.

5 Accepting and classifying results

5.1 Unconstrained problems

A walker's result x with returned (deflated) gradient $\hat{g}$ is a *local success* if

$$\hat{g} \text{ finite,} \quad \|\hat{g}\| < \varepsilon_g = 10^{-6}, \quad \lambda_* > \varepsilon_\lambda = 10^{-6}, \quad (10)$$

where λ are the eigenvalues of the true, symmetrized Hessian $\text{sym}(H(x))$ and λ_* is defined in (9). Since $\mathcal{D} \geq 1$, the condition on $\hat{g}$ implies $\|g\| < 10^{-6}$. Points that fail are returned with $r = 0$ and are not inserted into $\mathcal{L}$ (with one exception, Section 8.3).

5.2 Classification

An inserted point is classified from its gradient g and eigenvalues λ , in this order:

condition	class
$\max_i g_i > 10^{-3}$	degenerate
$\min_i \lambda_i < 10^{-5}$	zero curvature
all $\lambda_i > 0$	minimum
all $\lambda_i < 0$	maximum
mixed signs	saddle point

Only minima, maxima and saddle points are deflated.

5.3 The arrival test

A successful result x with gradient g is rejected as a duplicate if it lies inside the ball of a known deflation point:

$$\exists k : \|x - \hat{x}_k\| < r_k \text{ and } \max_i |g_i| < 10^{-5}. \quad (11)$$

The test is applied in the coordinator against the deflation set *at the time the result arrives*, which may be larger than the set the walker started with (Section 8).

6 Constraints

With constraints the walker always runs SLSQP. Its result generally lies on active constraints, where $g \neq 0$; the tests (10) would reject it. Constrained results are therefore judged by the Karush–Kuhn–Tucker (KKT) conditions [3].

6.1 Constraint representation

All forms accepted by `scipy.optimize.minimize` (`NonlinearConstraint`, `LinearConstraint`, dictionaries of type `eq/ineq`) are normalized into blocks $c^L \leq c(x) \leq c^U$; dictionary constraints become $c(x) \geq 0$ or $c(x) = 0$. Derivatives the user does not supply are approximated:

- J by central differences, step $h_i = 10^{-6} \max(1, |x_i|)$;
- the weighted Hessian $\sum_i w_i \nabla^2 c_i$ from the user’s `hess(x, w)` if given; else by central differences of $J^T w$ (symmetrized) if J is given; else by second differences of $w^T c$ with step $10^{-4} \max(1, |x_i|)$ — differencing a difference quotient would lose half the significant digits.

The box Ω is not part of the constraints.

6.2 First-order conditions: multipliers by bounded least squares

At x , with tolerances $\tau_i = 10^{-6} \max(1, |\text{bound}_i|)$:

$$\begin{aligned} \mathcal{E} &= \{i : c_i^L = c_i^U\} && \text{(equalities),} \\ \mathcal{A}^L &= \{i \notin \mathcal{E} : c_i^L \text{ finite, } c_i(x) - c_i^L \leq \tau_i\} && \text{(active lower bounds),} \\ \mathcal{A}^U &= \{i \notin \mathcal{E} : c_i^U \text{ finite, } c_i^U - c_i(x) \leq \tau_i\} && \text{(active upper bounds),} \end{aligned}$$

$\mathcal{A} = \mathcal{E} \cup \mathcal{A}^L \cup \mathcal{A}^U$, and x is *feasible* if every constraint is satisfied within τ_i . At a local minimum of (1), $g = J_{\mathcal{A}}^T w$ with $w_i \geq 0$ on $\mathcal{A}^L$ (moving into the feasible side would increase f), $w_i \leq 0$ on $\mathcal{A}^U$, and w_i free on $\mathcal{E}$. HGDL computes the multipliers from the true gradient,

$$w_{\mathcal{A}} = \arg \min_w \|J_{\mathcal{A}}^T w - g\| \quad \text{s.t.} \quad w_i \geq 0 \ (i \in \mathcal{A}^L), \quad w_i \leq 0 \ (i \in \mathcal{A}^U), \quad (12)$$

($w_i = 0$ for inactive constraints) with bounded-variable least squares [4], the same way for every local optimizer, and forms the gradient of the Lagrangian $L = f - w^T c$:

$$\nabla L(x) = g(x) - J(x)^T w. \quad (13)$$

The sign bounds in (12) matter: a point where g points out of the feasible set (so that f decreases into it) cannot be balanced with admissible multipliers and keeps a large ∇L . Without active constraints, $\nabla L = g$.

6.3 Second-order conditions and the radius

Let $\mathcal{S} = \mathcal{E} \cup \{i \in \mathcal{A} : |w_i| > 10^{-8}\}$ be the strongly active constraints and Z an orthonormal basis of the null space of $J_{\mathcal{S}}$ (the tangent space). The curvature that decides about a minimum is that of the Lagrangian along the tangent space,

$$\mu = \text{eig}\left(Z^T (\text{sym}(H) - \sum_i w_i \nabla^2 c_i) Z\right), \quad (14)$$

a vector of length $D - \text{rank } J_{\mathcal{S}}$. The result is accepted if it is feasible, $\|\nabla L\| < 10^{-6}$, and $\min \mu > 10^{-6}$; if the strongly active constraints fix x completely (Z empty, e.g. a vertex with positive multipliers), the first-order conditions suffice. The radius is (9) with $\lambda = \mu$, and the cap $\rho \|u - l\|$ when Z is empty. The reported df/dx is ∇L and the reported eigenvalues are μ , so the classification table above applies unchanged. Weakly active constraints ($w_i \approx 0$) are left out of $\mathcal{S}$, which makes the second-order test conservative.

6.4 Newton polish on the KKT system

SLSQP stops when f stops changing; near an optimum this leaves $\|\nabla L\|$ around the square root of its tolerance ($\approx 10^{-5}$), above ε_g . Before the test, HGDL therefore applies up to five Newton steps to the KKT system of the strongly active constraints with targets t_i (the bound each one sits on),

$$\begin{bmatrix} \nabla^2 L & -J_{\mathcal{S}}^T \\ J_{\mathcal{S}} & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta w \end{bmatrix} = - \begin{bmatrix} \nabla L \\ c_{\mathcal{S}}(x) - t_{\mathcal{S}} \end{bmatrix}, \quad (15)$$

solved in the least-squares sense. Only Δx is used: at the new point the active set and the multipliers are recomputed with (12). A step is kept only if the point stays feasible and the residual norm of the right-hand side decreases; the polish is attempted only at feasible points with $\|\nabla L\| < 10^{-3}$, so it refines nearly converged walkers and leaves stalled ones alone. Converging at the quadratic rate of Newton's method, one or two steps take $\|\nabla L\|$ from 10^{-5} to machine precision.

6.5 Constraints and deflation

Deflation is applied unchanged: SLSQP receives $G = \mathcal{D}g$. Mathematically a KKT point of the original problem remains a KKT point of the deflated one ($\mathcal{D}g = J^T(\mathcal{D}w)$); in practice SLSQP's line search on the undeflated f stalls inside the deflation ball, and these stalled walkers fail the KKT test. Walkers that do reach a known optimum are rejected by the arrival test (11), whose gradient condition uses ∇L . Constraints cannot be combined with stationary-point mode, since SLSQP is a minimizer.

7 Global step

New walkers start from batches of N_w points. Let $n = \min(|\mathcal{L}|, N_w)$ and let $X \in \mathbb{R}^{n \times D}$, $y \in \mathbb{R}^n$ be the positions and values of the n best entries of $\mathcal{L}$. The global method proposes n points, and the batch is padded with $N_w - n$ points drawn uniformly from Ω ; the very first batch is the user’s x^0 , padded or truncated in the same way.

Genetic step (default; Algorithm 5). Values are mapped to fitnesses that favour low f , $q_j = 2 - (y_j - \min y) / (\max y - \min y)$ (all $q_j = 1$ if y is constant), normalized to $p_j = q_j / \sum q$, and sharpened with the “unfairness” $\nu = 100$: $p_j \leftarrow p_j^{\nu-1} / \sum_i p_i^{\nu-1}$. Each child draws two parents m, d from p (with replacement) and is their fitness-weighted average plus a small Gaussian perturbation,

$$x_{\text{child}} = \frac{p_m X_m + p_d X_d}{p_m + p_d} + \xi, \quad \xi \sim \mathcal{N}(0, \text{diag}(\omega(u-l)^2)), \quad \omega = 5 \cdot 10^{-4}.$$

Children outside Ω are replaced by uniform samples. Children therefore start near (or between) the best known optima; deflation is what makes a walker started there move to a *different* optimum.

Random step. The n points are drawn uniformly from Ω .

Callable. A user function (`X`, `y`, `bounds`, `n`) returning n points.

8 Asynchronous execution

8.1 Coordinator and walkers

`optimize()` is non-blocking. It sends one `Problem` object (objective, derivatives, bounds, constraints, settings, run id) to every worker once, starts a coordinator thread in the calling process, and returns. Walkers are ordinary task tasks that receive a reference to that object, their start point and a copy of the deflation set; task places them on free worker threads and re-runs them if a worker dies. All state — the list $\mathcal{L}$ and the set of pending tasks — lives in the calling process and is changed only by the coordinator thread, under a lock that `get_latest()` also takes before returning a deep copy.

The coordinator (Algorithm 1) keeps N_w walkers in flight and consumes them in completion order. There is no barrier: a slow or diverging walker occupies only its own worker thread while all others continue (Figure 2). The number of walkers is re-read every N_w completions, so adaptive clusters may grow or shrink. The run ends when $N_{\text{epochs}} N_w$ walkers have completed; walkers still running are then cancelled or stopped, so `get_final()` never waits for a straggler.

8.2 Consistency between walkers and the changing deflation set

A walker never sees the live list. It receives a copy of the deflation points and radii when it is submitted and optimizes one fixed deflated function from start to end, so no walker observes the function change under it, and walkers share no mutable state. The price is staleness: a walker does not know optima found after it started and may converge to one of them. Uniqueness is therefore enforced where results arrive: the coordinator handles one completed walker at a time and applies the arrival test (11) against the *current* deflation set, so if two walkers find the same new optimum, the first to arrive is inserted and the second is rejected. (With an epoch barrier, all walkers of an epoch shared the snapshot taken at its start; a walker’s snapshot is now at least as recent.)

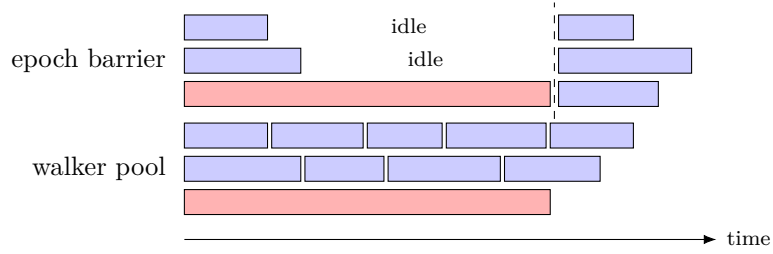


Figure 2: Three worker threads, one slow walker (red). With an epoch barrier (top) the other threads wait for it; in the walker pool (bottom) they keep starting new walkers.

8.3 The first results

If none of the first N_w completed walkers produced a successful result and $\mathcal{L}$ is still empty, their results are inserted anyway (they are classified, typically as degenerate, and not deflated). This guarantees that `get_final()[0]` exists, which downstream users rely on, e.g. when every optimum lies on the boundary of Ω .

8.4 Stopping and time limits

Between two iterations, every walker asks a local watch whether to give up:

- its `local_time_limit` (seconds, optional) has passed, or
- its run was stopped. `cancel_tasks()` and the end of the run cancel the pending tasks and send one message to every worker (`client.run`) that records the run id in a set in the worker's memory; walkers check that set. This costs one message per worker, never a request per walker, and therefore scales to thousands of walkers.

scipy optimizers are stopped from their callback (SLSQP and TNC let the resulting `StopIteration` escape; the walker then uses the last iterate), `dNewton` through a hook in its loop. A stopped walker's result is judged like any other and usually rejected. A single evaluation of f , g or H cannot be interrupted, and neither can a user-supplied local optimizer.

8.5 Reproducibility

All random numbers are drawn in the coordinator. With one worker the run is reproducible for a fixed seed; with several workers the completion order varies, so the set of optima found is stable but the run is not bit-for-bit reproducible.

9 Pseudocode

Algorithm 1 Coordinator (HGDL.optimize, _run_walkers)

Input: f, g, H , bounds $[l, u]$, constraints, start points x^0 , N_{epochs}

- 1: $N_w \leftarrow$ number of worker threads; $Q \leftarrow x^0$ padded/truncated to N_w rows
- 2: send $\mathcal{P} = (f, g, H, l, u, \text{constraints}, \text{settings}, \text{run id})$ to all workers
- 3: $\mathcal{L} \leftarrow \emptyset$; $n_c \leftarrow 0$; $F \leftarrow \emptyset$ $\triangleright F$: failed results among the first N_w
- 4: **start** N_w walkers $\triangleright$ each via SUBMIT
- 5: **for** each walker result $\mathcal{R}$, in order of completion **do**
- 6: **if** stopped **then break**
- 7: $n_c \leftarrow n_c + 1$
- 8: **if** $\mathcal{R}$ successful **and** not rejected by the arrival test against current $\mathcal{D}$ **then**
- 9: classify $\mathcal{R}$, insert into $\mathcal{L}$ (sorted by f)
- 10: **else if** $n_c \leq N_w$ **then**
- 11: $F \leftarrow F \cup \{\mathcal{R}\}$
- 12: **end if**
- 13: **if** $n_c = N_w$ **and** $\mathcal{L} = \emptyset$ **then**
- 14: insert all of F into $\mathcal{L}$ $\triangleright$ Section 8.3
- 15: **end if**
- 16: **if** $n_c \geq N_{\text{epochs}}N_w$ **then break**
- 17: **if** $n_c \bmod N_w = 0$ **then** re-read N_w
- 18: start walkers until N_w are in flight
- 19: **end for**
- 20: cancel pending walkers; mark the run stopped on every worker
- 21: **procedure** SUBMIT
- 22: **if** $Q = \emptyset$ **then**
- 23: $Q \leftarrow \text{GLOBALSTEP}(\text{best min}(|\mathcal{L}|, N_w) \text{ entries of } \mathcal{L})$ padded with uniform samples to N_w
- 24: **end if**
- 25: pop x_s from Q ; start WALKER($x_s, \mathcal{P}$, copy of $\mathcal{D}$) as a task
- 26: **end procedure**

Algorithm 2 Walker (local_method)

Input: start x_s , problem $\mathcal{P}$, deflation set $\{(\hat{x}_k, r_k)\}$

```
1:  $G \leftarrow \mathcal{D}g$ ,  $J_G \leftarrow \mathcal{D}H + g\nabla\mathcal{D}^T$  ▷ Eqs. (5)–(7)
2: if optimizer is dNewton then
3:    $x \leftarrow \text{DEFLATEDNEWTON}(x_s, \text{saddle-free} = (\text{mode is minimization}))$ 
4: else
5:    $x \leftarrow \text{scipy minimize}(f, x_s, \text{jac} = G, \text{hess} = J_G, \text{bounds}, \text{constraints}, \text{callback}=\text{watch})$  or
     user callable
6: end if
7: if constraints then
8:    $x \leftarrow \text{NEWTONPOLISH}(x)$  ▷ Algorithm 4
9:    $(\hat{g}, \lambda, \text{feasible}) \leftarrow \text{KKT}(x)$  ▷  $\hat{g} = \nabla L$ ,  $\lambda = \mu$ 
10: else
11:    $\hat{g} \leftarrow$  gradient returned by the optimizer;  $\lambda \leftarrow \text{eig}(\text{sym } H(x))$ ;  $\text{feasible} \leftarrow \text{true}$ 
12: end if
13:  $\lambda_* \leftarrow \min \lambda$  (minimization) or  $\min |\lambda|$  (stationary points);  $\lambda_* \leftarrow \infty$  if  $\lambda$  is empty
14:  $\text{success} \leftarrow \text{feasible} \wedge \hat{g} \text{ finite} \wedge \|\hat{g}\| < 10^{-6} \wedge \lambda_* > 10^{-6}$ 
15:  $r \leftarrow \min(1/\lambda_*, 0.1 \|u - l\|)$  if success, else 0
16: return  $(x, f(x), \hat{g}, \lambda, r, \text{success})$ 
```

Algorithm 3 Deflated Newton (DNewton)

Input: x , G , J_G , bounds, tolerance t , iteration limit N , saddle-free flag, watch

```
1:  $s \leftarrow \infty$ ,  $k \leftarrow 0$ 
2: while  $\|s\|_\infty > t$  or  $\|G(x)\|_\infty > t$  do
3:    $x \leftarrow \Pi_{[l,u]}(x)$  ▷ projection onto the box
4:   if saddle-free then
5:      $s \leftarrow \text{step (2)}$ 
6:   else
7:      $s \leftarrow -J_G(x)^{-1}G(x)$  ▷ least squares if singular
8:   end if
9:   if  $s$  not finite then return  $x$ , failure
10:   $x \leftarrow x + s$ ;  $k \leftarrow k + 1$ 
11:  if  $k > N$  or watch says stop then return  $x$ , failure
12: end while
13: return  $x$ , converged
```

Algorithm 4 KKT test and Newton polish (kkt_conditions, newton_polish)

```
1: function STATE( $x$ )
2:   evaluate  $c(x)$ ,  $J(x)$ ; determine  $\mathcal{E}, \mathcal{A}^L, \mathcal{A}^U$  and feasibility
3:    $w \leftarrow$  bounded least squares (12) on the true  $g(x)$ 
4:    $\mathcal{S} \leftarrow \mathcal{E} \cup \{i \in \mathcal{A} : |w_i| > 10^{-8}\}$ 
5:   return  $c, J, w, \mathcal{S}$ , feasible
6: end function
7: function KKT( $x$ )
8:   ( $c, J, w, \mathcal{S}$ , feasible)  $\leftarrow$  STATE( $x$ );  $\nabla L \leftarrow g - J^T w$ 
9:   if  $\text{null}(J_{\mathcal{S}}) = \{0\}$  then
10:    return  $\nabla L, \emptyset$ , feasible
11:   end if
12:    $Z \leftarrow$  orthonormal basis of  $\text{null}(J_{\mathcal{S}})$ 
13:   return  $\nabla L$ , eig( $Z^T(\text{sym } H - \sum_i w_i \nabla^2 c_i)Z$ ), feasible
14: end function
15: function NEWTONPOLISH( $x$ )
16:   ( $c, J, w, \mathcal{S}$ , feasible)  $\leftarrow$  STATE( $x$ )
17:   if not feasible or  $\|g - J^T w\| \geq 10^{-3}$  then return  $x$ 
18:   for at most 5 steps, while residual  $> 10^{-12}$  do
19:     solve (15) for  $\Delta x$  (least squares);  $x' \leftarrow x + \Delta x$ 
20:     if  $x'$  infeasible or its residual is not smaller then break
21:      $x \leftarrow x'$ ; recompute STATE( $x$ )
22:   end for
23:   return  $x$ 
24: end function
```

Algorithm 5 Genetic step (genetic_step)

Input: parents $X \in \mathbb{R}^{n \times D}$, values y , bounds, number of children n

```
1:  $q_j \leftarrow 2 - (y_j - \min y) / (\max y - \min y)$   $\triangleright$  all 1 if  $y$  is constant
2:  $p \leftarrow q / \sum q$ ;  $p \leftarrow p^{99} / \sum p^{99}$   $\triangleright$  unfairness  $\nu = 100$ 
3: for each child do
4:   draw parents  $m, d \sim p$ 
5:    $x \leftarrow (p_m X_m + p_d X_d) / (p_m + p_d) + \mathcal{N}(0, \text{diag}(5 \cdot 10^{-4}(u - l))^2)$ 
6:   if  $x \notin \Omega$  then  $x \leftarrow$  uniform sample from  $\Omega$ 
7: end for
```

10 Implementation map

concept	code
user API, coordinator, walker pool, stopping	hgdl/hgdl.py: HGDL, optimize, _run_walkers, _submit_walker, _accept, _stop_walkers
problem snapshot sent to the workers	hgdl/problem.py: Problem; default Hessian approximate_hessian
walker, acceptance, radius, watch	hgdl/local_methods/local_optimizer.py: local_method, _Watch, stop_run
arrival test	local_optimizer.py: collect_results
bump, deflation operator, G , J_G	hgdl/local_methods/bump_function.py: b, _bumps, deflation_and_gradient, deflated_grad, deflated_hess
deflated Newton, saddle-free step	hgdl/local_methods/dNewton.py: DNewton, saddle_free_step
constraints: normalization, KKT, polish	hgdl/local_methods/kkt.py: normalize_constraints, kkt_conditions, newton_polish
list, classification, deflation set	hgdl/optima.py: optima.fill_in_optima_list, get_deflation_points
global step	hgdl/global_methods/global_optimizer.py: run_global, genetic_step, random_step

11 Constants

quantity	value	where
gradient threshold for success ε_g	10^{-6}	local_method
curvature threshold ε_λ	10^{-6}	local_method
radius cap ρ (fraction of the domain diagonal)	0.1	MAX_RADIUS_FRACTION
bump cap $\beta_{\max}$	$1 - 10^{-5}$	BUMP_CAP
classifier: degenerate if $\max g_i >$	10^{-3}	fill_in_optima_list
classifier: zero curvature if $\min \lambda_i <$	10^{-5}	fill_in_optima_list
arrival test: gradient below	10^{-5}	collect_results
dNewton iteration limit (default)	1000	DNEWTON_DEFAULT_MAX_ITER
dNewton/scipy tolerance (default)	10^{-10}	optimize(tolerance=)
saddle-free eigenvalue floor	$10^{-8} \max(1, \max \theta)$	saddle_free_step
constraint activity/feasibility tolerance	$10^{-6} \max(1, \text{bound})$	ACTIVE_TOL
strongly active multiplier	$ w_i > 10^{-8}$	MULTIPLIER_TOL
constraint finite-difference steps	$10^{-6}, 10^{-4}$ (relative)	FD_STEP, FD2_STEP
Newton polish: attempted below / steps	$\ \nabla L\ < 10^{-3} / 5$	newton_polish
default Hessian forward-difference step	10^{-6}	approximate_hessian
genetic unfairness ν / perturbation ω	100 / $5 \cdot 10^{-4}$	genetic_step

12 Differences from HGDN and open issues

Deliberate differences from [1]. The bump has radial support ($\alpha = r^2$) instead of a per-coordinate one; the deflation operator is a product rather than a sum; the genetic step averages two parents with fitness weights plus a small perturbation instead of gene crossover and mutation; and the inner loop of the paper’s Algorithm 4 (reset the walkers and repeat the deflated search until nothing new is found) is not implemented — each walker performs one local search and is replaced.

Deflation and the saddle-free step. As shown in Section 4.4, the saddle-free Newton step used in minimization mode is a descent direction for f and is not repelled by deflation. A preliminary experiment (not implemented in `hgd1`) replaces it near deflated points by

$$s = -(|H| + g \nabla \ln \mathcal{D}^T)^{-1} g,$$

taking the absolute value of the eigenvalues of the true Hessian only. Far from all deflation points ($\nabla \ln \mathcal{D} = 0$) this is the saddle-free step; near a deflated minimum ($|H| = H$) it is the plain Newton step of Section 4.3, which moves away. In the experiment of Table 1 only 2 of 100 walkers returned to the deflated minimum and 24 reached other minima, but most did not converge within 200 iterations; the step needs further work before it can be adopted.

Bounds. Optima on the boundary of Ω are not accepted, since Ω is not treated as a constraint (Section 8.3 keeps such runs from returning nothing).

Communication. Every walker carries its copy of the deflation set; with 10^4 optima in high dimension this is megabytes per task. Scattering the set once and passing references is a possible later optimization.

References

- [1] M. M. Noack and S. W. Funke. Hybrid genetic deflated Newton method for global optimisation. *Journal of Computational and Applied Mathematics*, 325, 2017.
- [2] Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances in Neural Information Processing Systems 27*, 2014.
- [3] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006.
- [4] P. B. Stark and R. L. Parker. Bounded-variable least-squares: an algorithm and applications. *Computational Statistics*, 10:129–141, 1995.